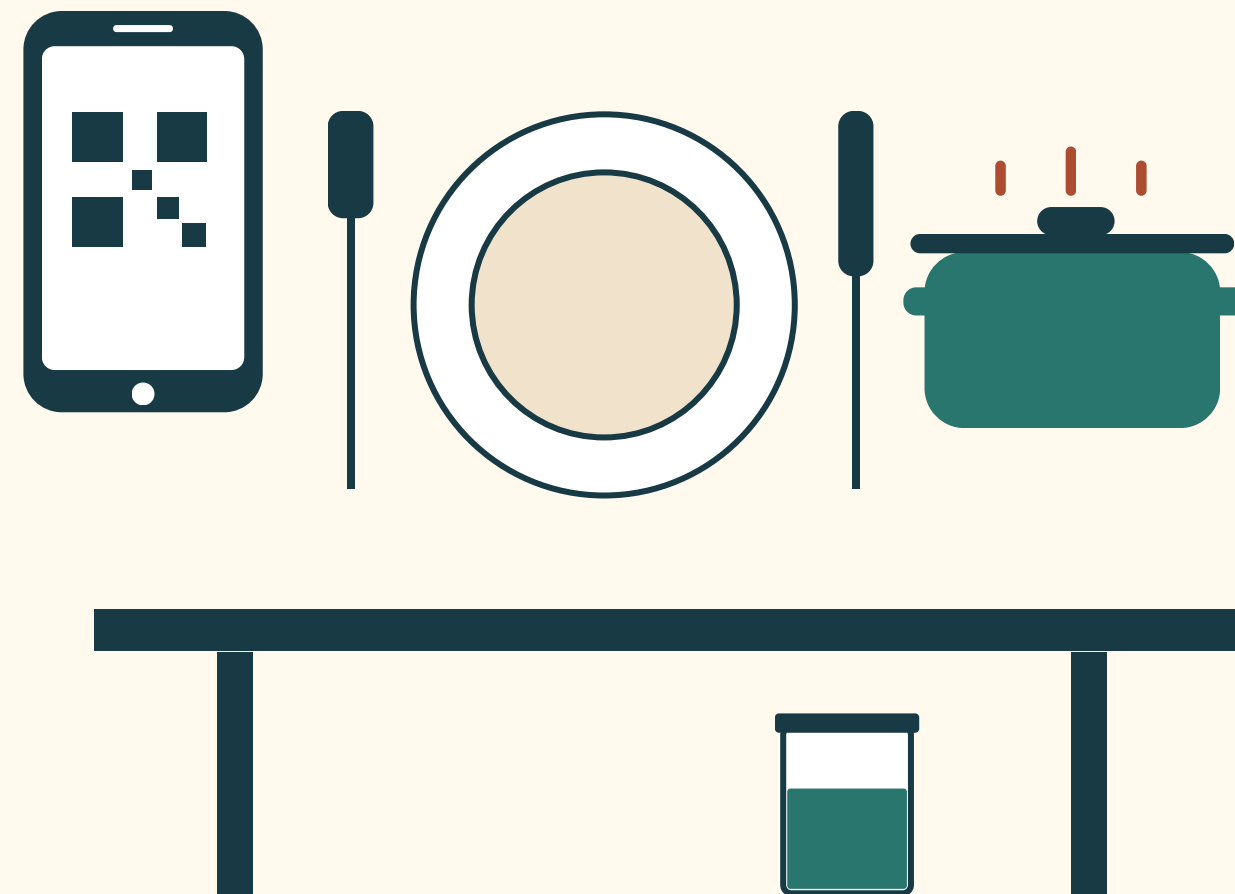


# ระบบจัดการ ร้านบุฟเฟต์

CP353002

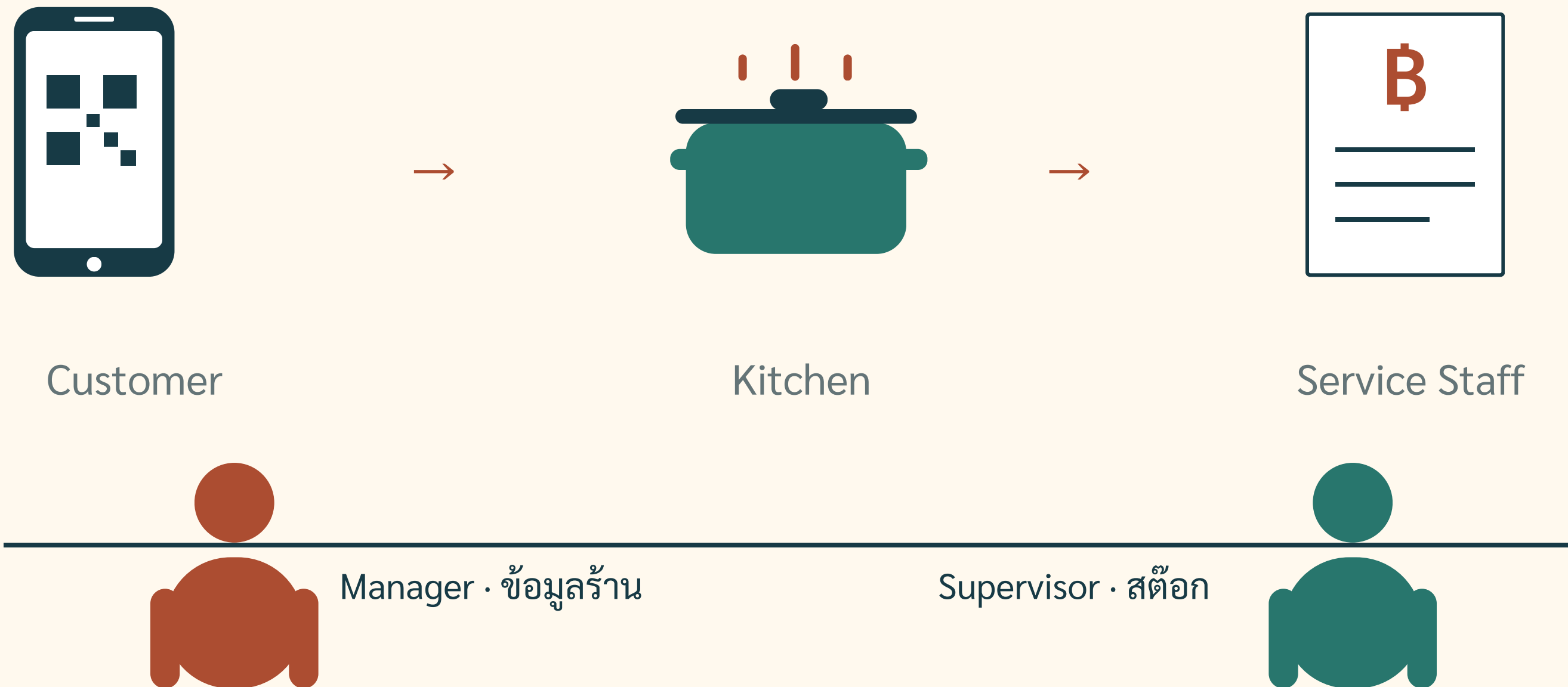
ทีม 5 คน · 9 นาที

โต๊ะ → ออเดอร์ → ครั้ว → สต็อก → บิล



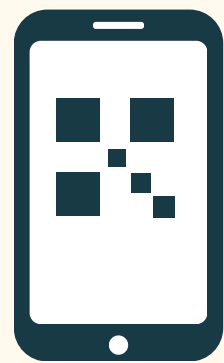
เชื่อมการเปิดโต๊ะและการสั่งอาหารเข้ากับรอบเดียวกัน  
ติดตามงานครั้ว การใช้วัตถุดิบ และการชำระเงินจนปิดรอบ

# หนึ่งรอบโต๊ะ เชื่อมทุกบทบาท



ลูกค้าสั่งอาหาร คราวเริ่มทำ และพนักงานดูแลการชำระเงิน  
แต่ละบทบาทใช้ข้อมูลรอบโต๊ะเดียวกัน โดย backend ตรวจสอบสิทธิ์ทุกคำขอ

# แยกหน้าจอ กฎธุรกิจ และข้อมูล



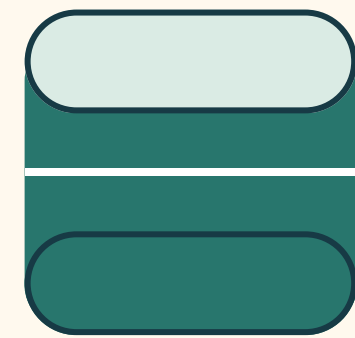
React



Controller



Service



Repository / DB

HTTP / DTO

กฎ + Transaction

SRP

ISP

DIP

Controller รับคำขอ ส่วน Service คำนวณกฎธุรกิจ  
Repository ดูแลการอ่านเขียนข้อมูล ทำให้ทดสอบแต่ละชั้นแยกกันได้

# ออเดอร์เก่า ยังคงใช้สูตรเดิม

Order #7

$0.100 \times 2$



0.200 kg

แก้สูตร

Order #11

$0.250 \times 2$

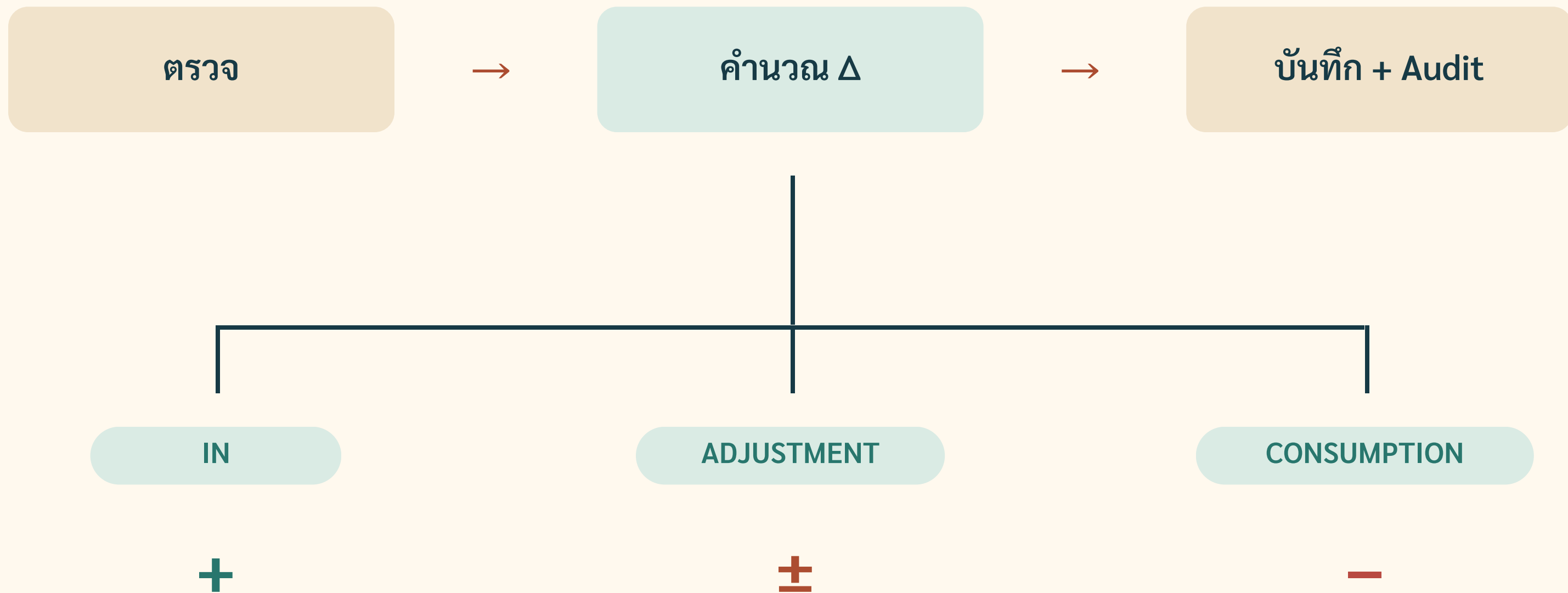


0.500 kg

Snapshot ตอนสั่ง · ตัวอย่างจาก UAT รุ่น af2b45b

ระบบเก็บสูตรตอนสั่งไว้กับออเดอร์  
เมื่อแก้สูตร ออเดอร์เก่าก็ยังหักวัตถุดิบตามสูตรเดิม

# Template Method ใช้ขั้นตอนร่วมกัน



Template กำหนดขั้นตอนตรวจและบันทึกไว้ร่วมกัน  
แต่ละ Processor เปลี่ยนวิธีคำนวณยอดและประเภทประวัติผ่าน Hook

# เริ่มทำ จิ้งห้กสต็อกครั้งเดียว

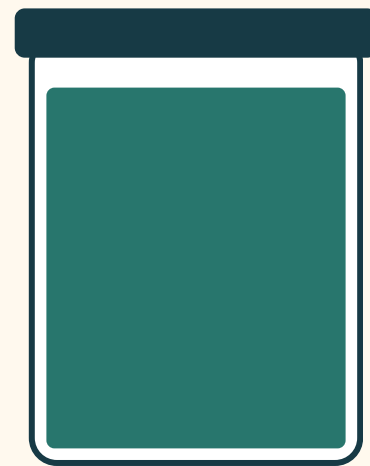
RECEIVED



PREPARING



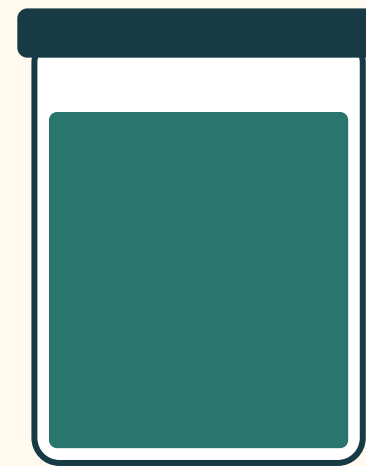
READY → SERVED



3.000



-0.200 kg



2.800

409

ไม่พอ → คงเดิม

UAT af2b45b · สูตร 0.100 × 2 · duplicate ไม่เพิ่ม CONSUMPTION

ระบบหักวัตถุดิบตามสูตรเมื่อเปลี่ยนเป็น PREPARING  
ถ้าสต็อกไม่พอ จะปฏิเสธทั้งคำขอและคงยอดเดิม

# Strategy แยกราคาและส่วนลด

BillCalculationStrategy



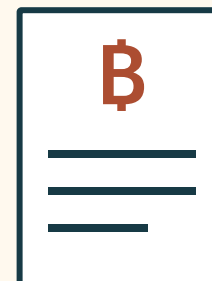
DiscountCalculationStrategy

ราคา

ส่วนลด



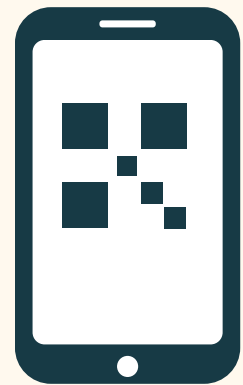
BigDecimal



HALF\_UP · 2

BillingEngine เรียกการคิดราคาและส่วนลดผ่าน Interface  
จึงเปลี่ยนวิธีคำนวณได้ โดยยังตรวจสอบและปิดเศษที่ส่วนกลาง

## เอกสาร API ตรงกับผลจริง



POST



201

Created



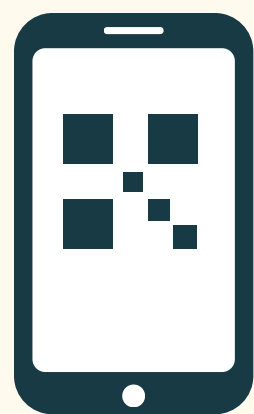
DTO

Location

PR52 · ผ่าน local / CI · public retest ของการแก้ไขยังไม่ยืนยัน

PR52 ประกาศ 201 พร้อม DTO และ Location ให้ตรงกับ API จริง  
และแก้ตัวอย่าง ErrorResponse ที่เคยแสดงข้อมูลข้าม Operation

## Live Demo · 3:30



เปิดโต๊ะ + QR

0:00–0:55

สั่ง · ครว · สต็อก

0:55–2:15

ชำระ + ปิดรอบ

2:15–3:30

[buffet-restaurant-management.onrender.com](https://buffet-restaurant-management.onrender.com) · เตรียมข้อมูลและซ้อมก่อนใช้งานจริง

สาธิตหนึ่งรอบโต๊ะ ตั้งแต่เปิด QR จนชำระเงินและปิดรอบ  
ระหว่างทางแสดงว่าสต็อกลดลงตอนครัวเริ่มทำอาหาร

# กฎชุด ทดสอบได้ หลักฐานตามรุ่น



สัญญาาร่วม



Regression



Revision + Environment

ใช้สัญญาาร่วมและ Regression ลดปัญหาระหว่างโมดูล  
ระบุรุ่นโค้ดและสภาพแวดล้อมของผลทดสอบ เพื่อให้ตรวจสอบย้อนกลับได้

# หลักฐานคนละรุ่น ต้องอ่านแยกกัน

PUBLIC

af2b45b

8 UI  
12 API

CI

25c7a40

431

Backend

187 + 6 Frontend

MERGED CODE

855a954

PR52

Swagger

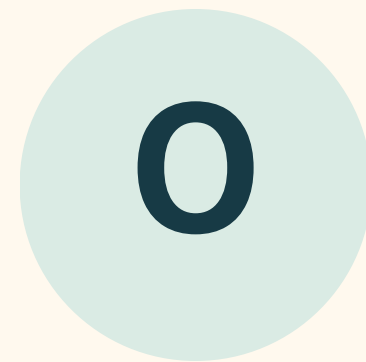
UI load-failure ไม่ได้รัน · Swagger public retest ยังไม่ยืนยัน

UAT แสดงผลบนเว็บจริง ส่วน CI ตรวจโค้ดในรุ่นที่ระบุ  
การ merge ยืนยันว่าโค้ดรวมแล้ว ยังต้องทดสอบการแก้ Swagger บนเว็บจริง

# SOLID ผ่านหน้าที่และสัญญา



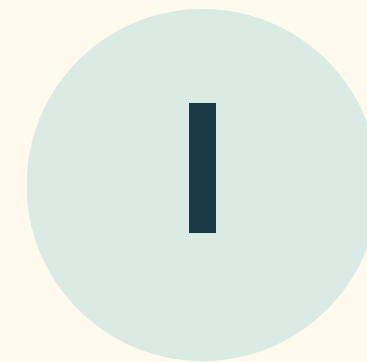
แยกหน้าที่



เพิ่มผ่าน Hook



รักษาสัญญา



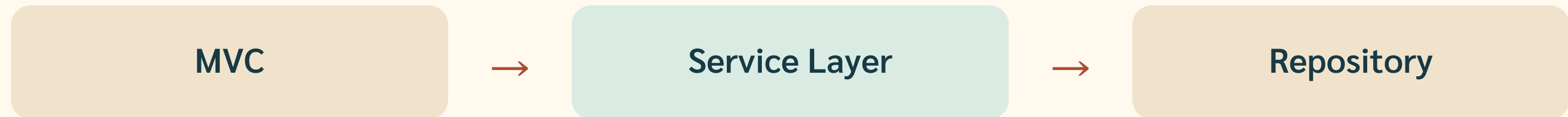
Interface พอดี



ฉีด Dependency

SOLID ช่วยแบ่งหน้าที่และจำกัดสิ่งที่แต่ละส่วนต้องรู้  
จึงเพิ่มหรือทดแทนพฤติกรรมได้ โดยรักษาสัญญาที่ส่วนอื่นเรียกใช้

# Enterprise Patterns ทำงานร่วมกัน



---

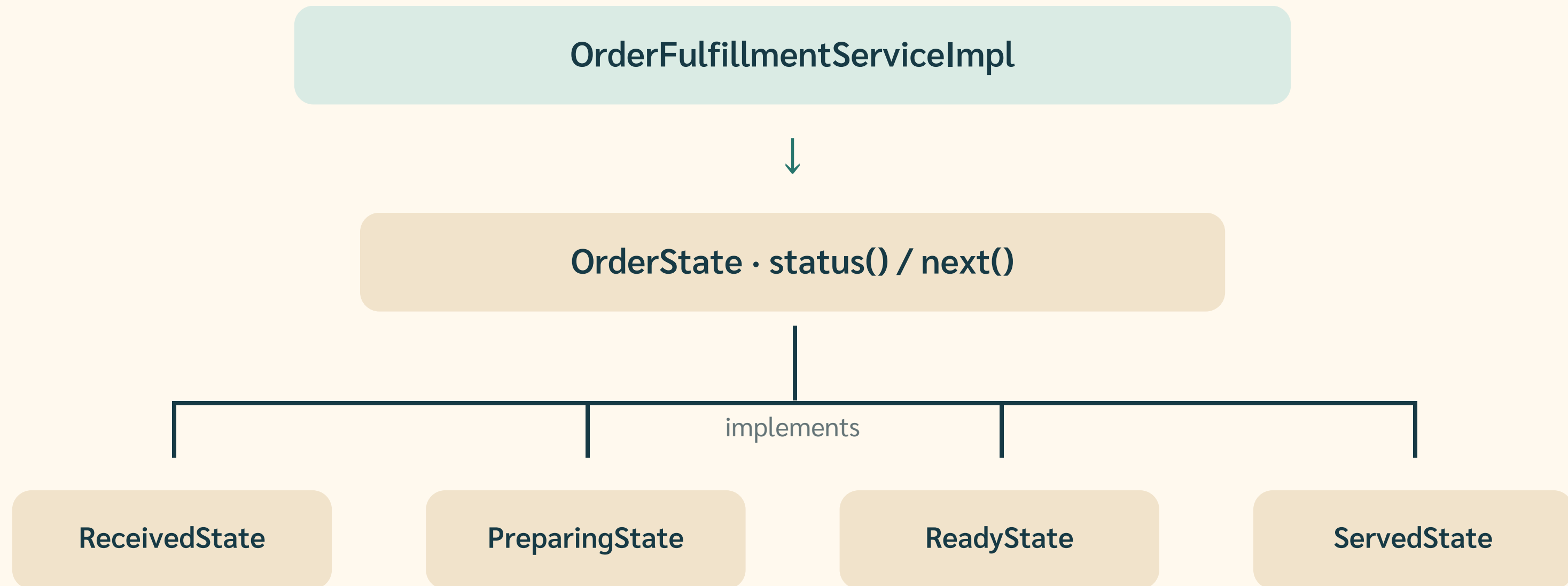
## Layered Architecture

DTO + Mapper

Dependency Injection

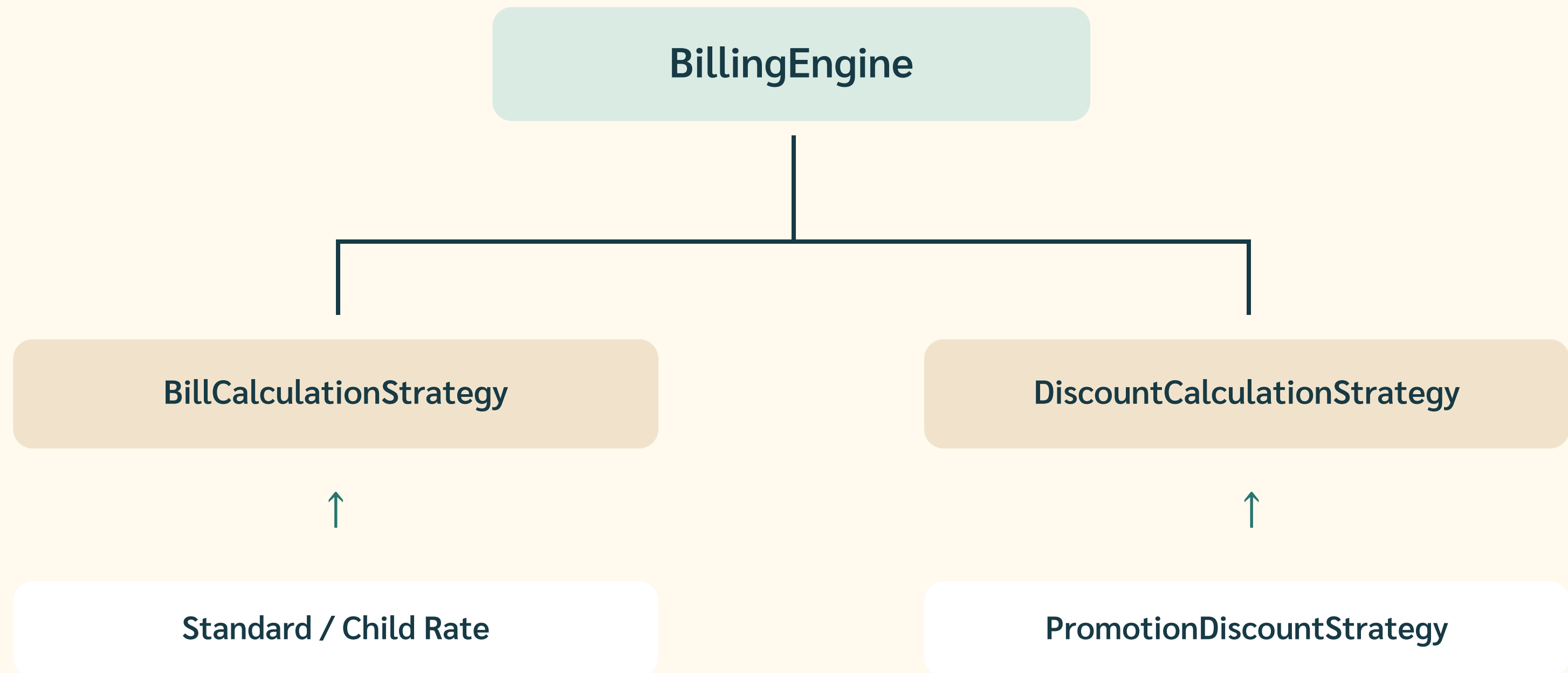
แต่ละชั้นรับผิดชอบงานของตนเอง และส่งข้อมูลผ่าน DTO  
Service ประสาน use case ส่วน Repository ดูแลการเข้าถึงฐานข้อมูล

## State · เลือกพฤติกรรมตามสถานะ



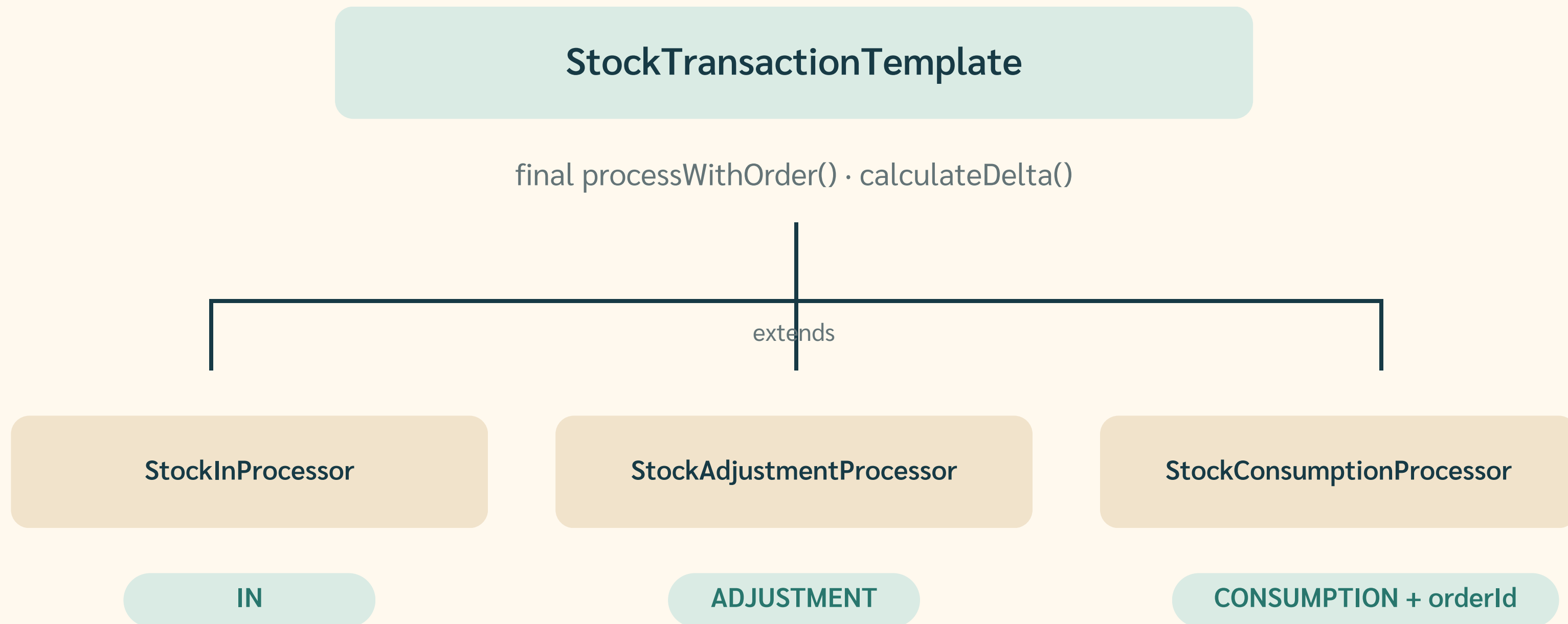
Service เลือก State จากสถานะปัจจุบันของออเดอร์  
แต่ละ State กำหนดสถานะถัดไป จึงปฏิเสธการข้ามขั้นหรือทำซ้ำ

# Strategy · Engine รู้เพียง Interface



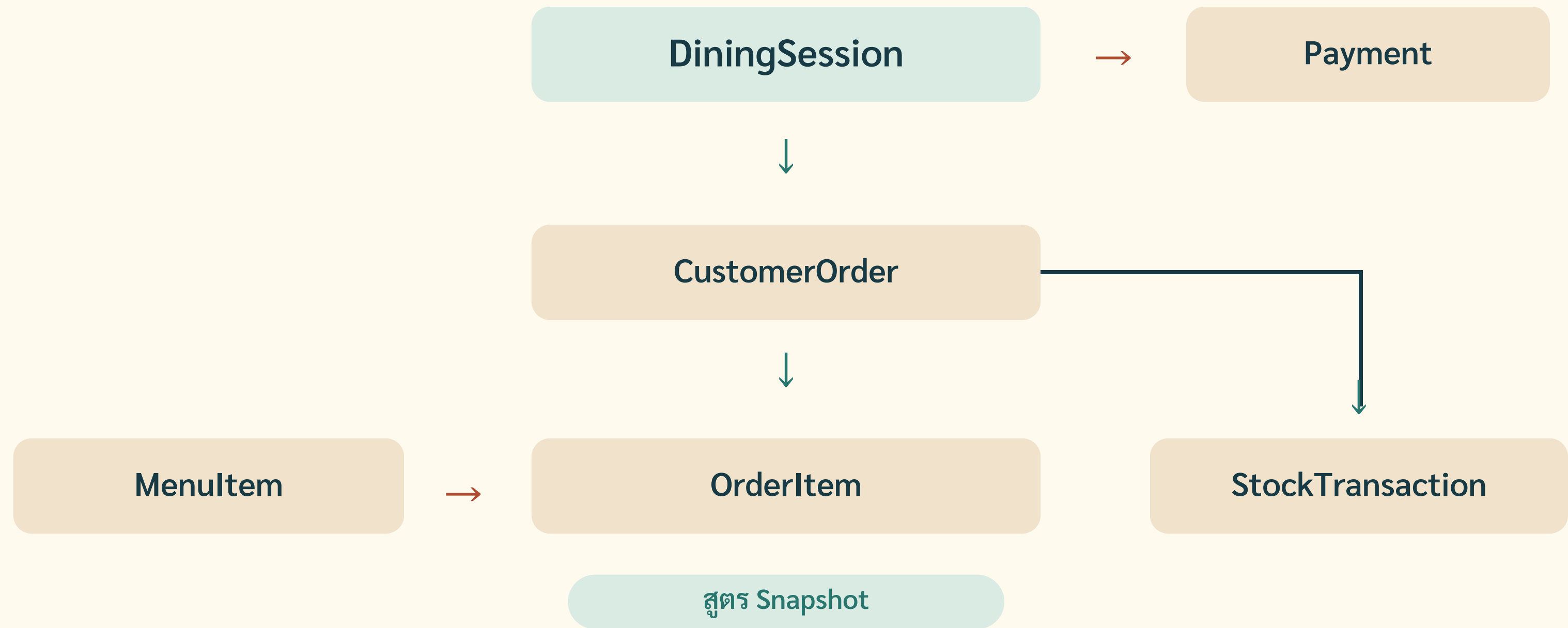
Engine ฟังพา Interface สำหรับราคาและส่วนลด  
จึงใช้วิธีคำนวณหลายแบบได้ โดยไม่ผูกกับคลาสใดคลาสหนึ่ง

# Template · เพิ่ม Consumption ผ่าน Hook



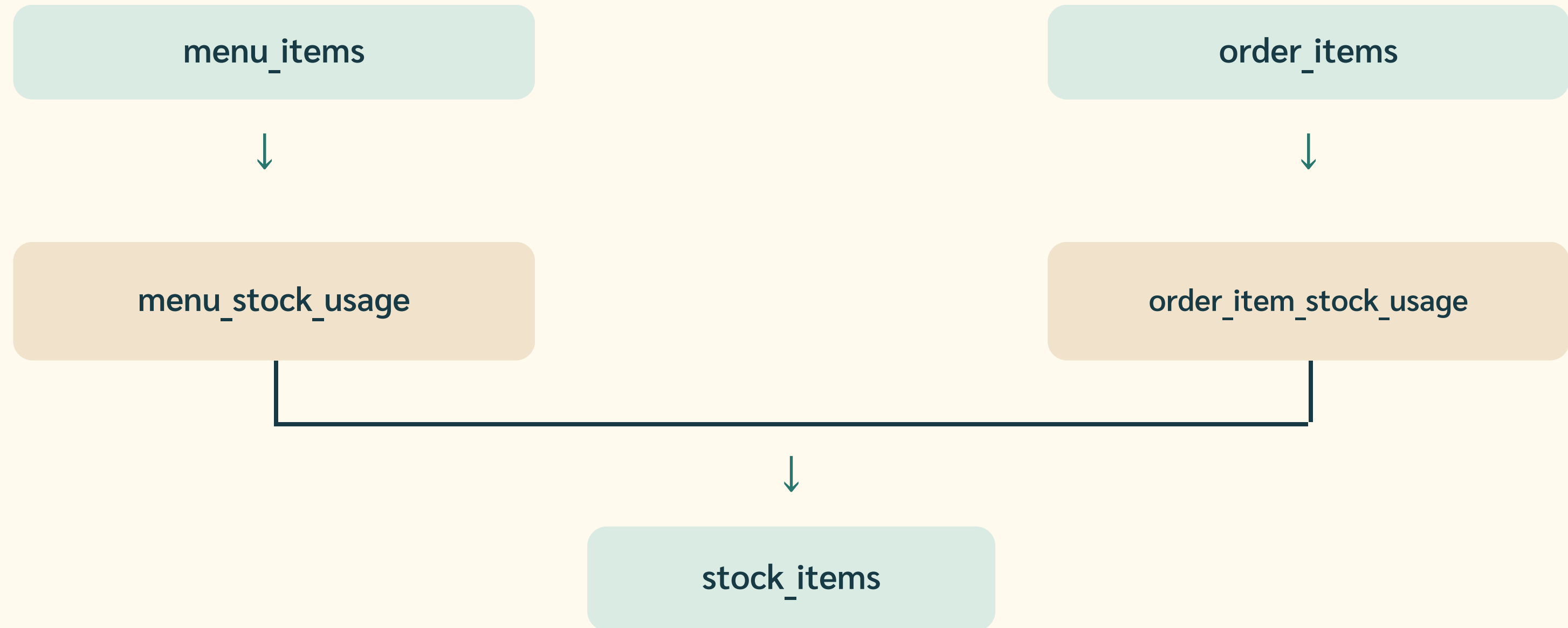
Template วางขั้นตอนตรวจสอบและบันทึกไว้เป็น final  
Consumption เพิ่มการหักยอดและผูกประวัติกับ orderId ผ่าน Hook

## Domain · รอบโต๊ะเป็นแกนกลาง



รอบโต๊ะเชื่อมออเดอร์กับการชำระเงิน ส่วน OrderItem เก็บสูตรตอนสั่ง  
ประวัติ CONSUMPTION อ้าง orderId เพื่อย้อนดูที่มาของการหักสต็อก

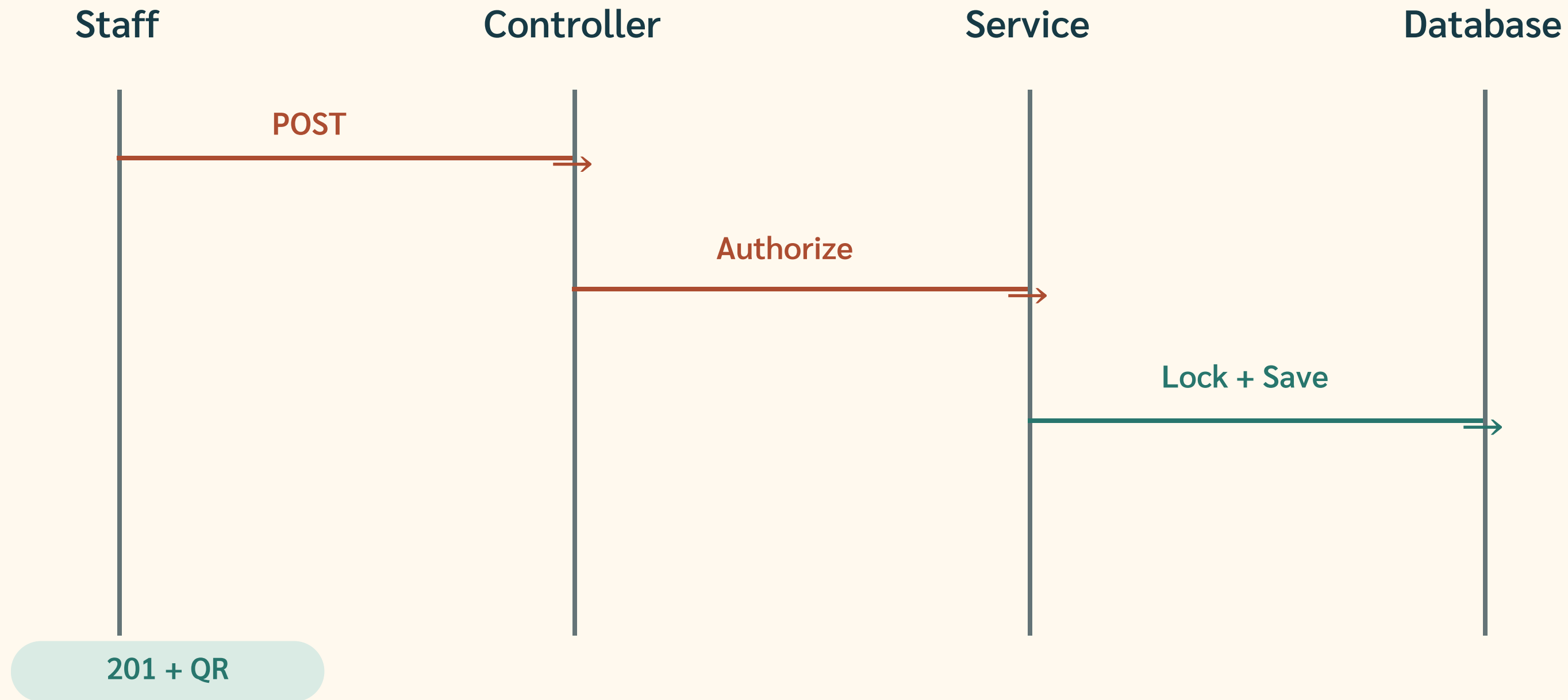
# ER · สูตรปัจจุบันแยกจาก Snapshot



V19 · FK + composite PK · Snapshot เป็นข้อมูลของ OrderItem

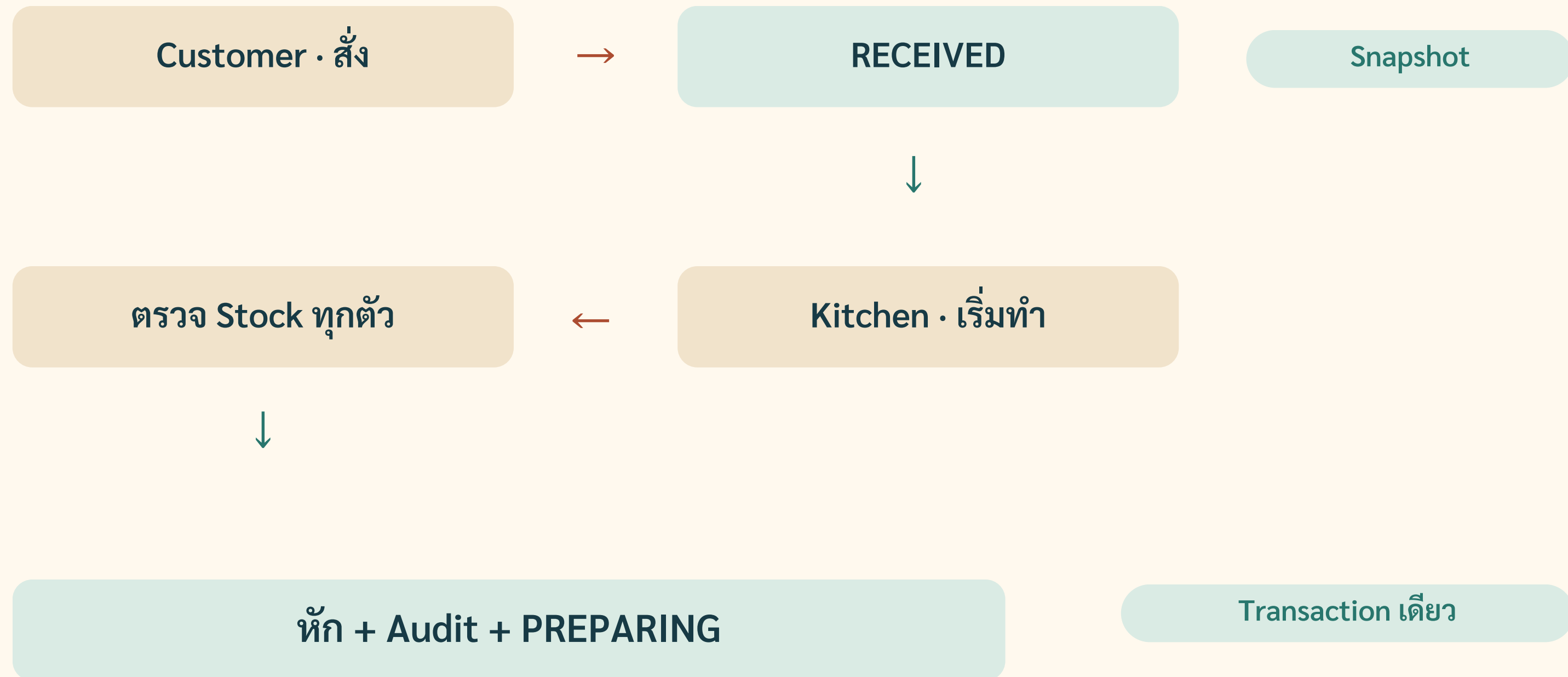
สูตรของเมนูปัจจุบันและสูตรตอนสั่งอยู่คนละตาราง  
FK และ composite PK รักษาการอ้างอิง โดยไม่ให้เกิดการแก้ไขเมนูเปลี่ยนออเดอร์เก่า

# Sequence · เป็ครอบอย่างเป็นขั้นตอน



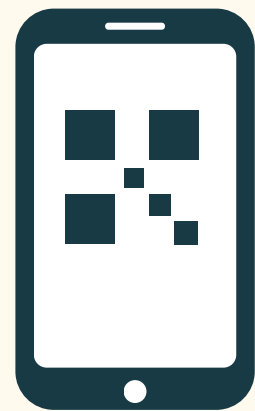
Service ตรวจสอบสิทธิ์และเงื่อนไข แล้วล็อกไ้ะก่อนสร้างรอบ  
ระบบบันทึกรอบกับสถานะไ้ะใน Transaction เดียวกัน และคืน 201 พร้อม QR

# Sequence · สั่งก่อน หักเมื่อเริ่มทำ



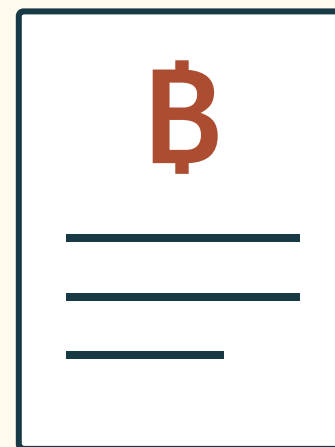
ตอนสั่ง ระบบสร้าง RECEIVED พร้อมสูตร Snapshot  
เมื่อครัวเริ่มทำ จึงตรวจทุกวัตถุดิบแล้วหักยอดพร้อมเปลี่ยนสถานะใน Transaction เดี่ยว

## Sequence · PAID แล้วจึงปิดรอบ



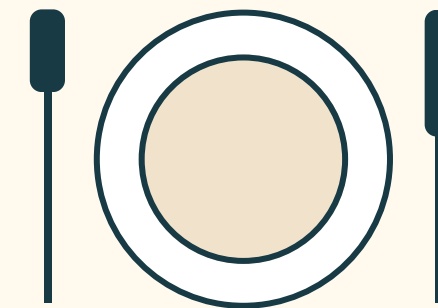
ขอคิดบิล

Customer



PAID

Service Staff



ปิดรอบ

AVAILABLE

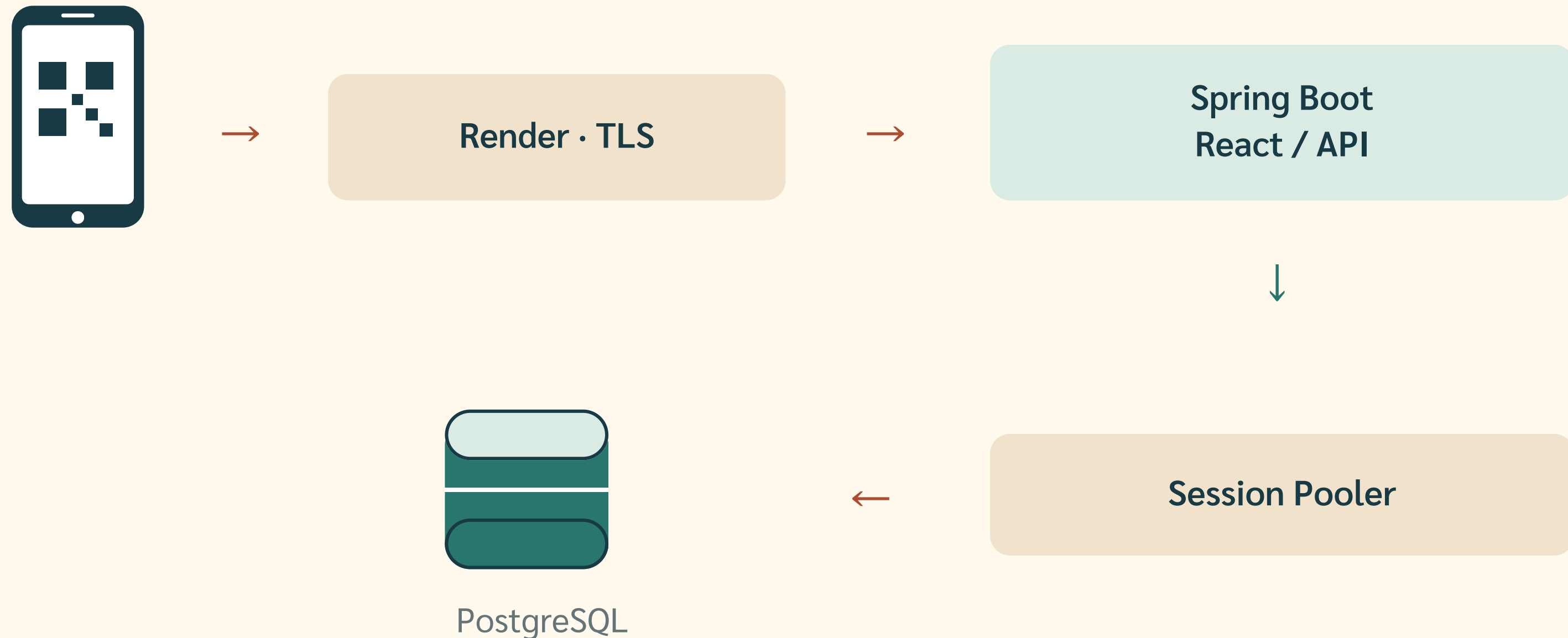
PAID ยืนยันการรับชำระ ส่วนการปิดรอบเป็นอีกคำสั่งหนึ่ง  
พนักงานปิดรอบหลังชำระแล้ว เพื่อคืนโต๊ะ AVAILABLE และยกเลิกสิทธิ์ลูกค้า

## Activity · ลูกค้าสั่งและติดตาม



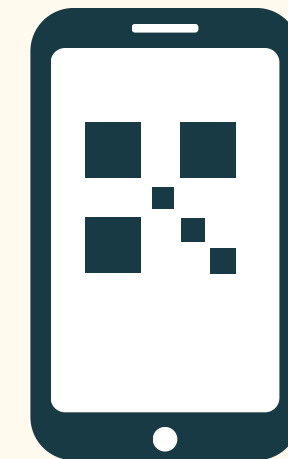
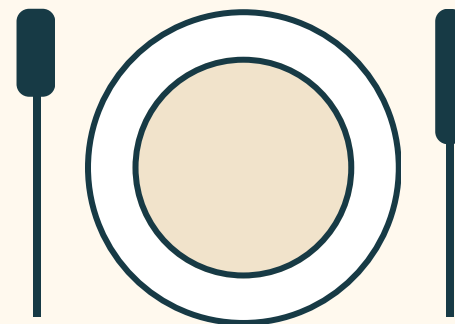
เมื่อขอคิดบิล ระบบหยุดรับออเดอร์เพิ่ม และเมื่อปิดรอบ สิทธิ์ลูกค้าสิ้นสุด  
หน้าเว็บแยกสิทธิ์หมดอายุออกจาก Network หรือ 5xx ที่ให้ลองใหม่ได้

# Deployment · API คุมการเข้าถึงข้อมูล



Browser เรียก API ผ่าน HTTPS แล้ว backend จึงติดต่อฐานข้อมูลผ่าน Pooler  
ภาพแสดงโครงสร้างที่ตั้งค่าไว้ ต้องยืนยันรุ่นที่ deploy ก่อนสาธิตจริง

## Use Case · เปิดโต๊ะ



SERVICE\_STAFF

AVAILABLE

ACTIVE + QR

ผัดเงื่อนไข → ปฏิสัธ · รายละเอียด / แหลงอ้างอิงอยูใน Notes

พนักงานเปิดได้เฉพาะโต๊ะ AVAILABLE ที่ผ่านเงื่อนไข  
ระบบสร้างรอบ ACTIVE พร้อม QR สำหรับให้ลูกค้าแลกสิทธิของรอบนี้